Skeletonize an Existing Repository

Skeletonizing a repository standardizes our development setup and environment, and it enables our Lineage GitHub Action to keep the repository updated and standardized.

About

Skeleton projects contain licensing information, as well as pre-commit hooks and GitHub Actions configurations appropriate for the major languages that we use. This lets us standardize cisagov GitHub projects to a list of cisagov skeleton projects.

General overview

The general outline of how to add a skeleton to a repository is:

1. Add the skeleton as a remote to the non-skeletonized repository
2. Pull with `--allow-unrelated-histories`
3. Fix all the inevitable conflicts
4. Review non-conflicting changes to prevent merging destructive upstream changes
5. Update skeleton's `example` references
6. Set up pre-commit
7. Fix additional problems that may arise
8. Make a pull request

Add a skeleton as remote

First, decide which of the available skeletons fits your existing repository. To see a list of available skeletons, use the `skeleton list` command or see the list of skeletons.

As an example, we'll be using `skeleton-python-library` in this document.

```
cd <target_repository>
git remote add skeleton-parent git@github.com:cisagov/skeleton-python-
library.git

# You can verify the remote has been added by
git remote --verbose

# Create a new branch for this work
git checkout -b skeletonize

# Pull skeleton's history
git pull skeleton-parent develop --allow-unrelated-histories
```

Set up your environment and pre-commit hooks

Follow the instructions in CONTRIBUTING.md on setting up pre-commit to run `setup-env` and enable the `pre-commit` hooks. If you have already set up the prerequisites, this involves:

```
# In the root directory of the repository
./setup-env
pre-commit install
```

Fix merge conflicts

This merge process will almost certainly fail, resulting in merge conflicts. The next step is to fix those conflicts and add the files once the fixes are in place.

```
# Determine which files need fixes
git status

# After fixing the merge conflicts in a file,  add it
git add <filename>

...
# When all conflicts have been fixed and added, commit to complete the
merge
# Remember to add a descriptive and useful commit message
git commit
```

Review non-conflicting changes

You don't only have to fix merge conflicts. It is important to also look at the unconflicted changes listed in the outputs of `git status` and `git diff origin/develop` and verify that you want to include all those changes.

```
git diff origin/develop
```

This step is often overlooked because it is rarely needed, but it can save you from merging in destructive upstream changes.

Update skeleton's `example` references

This step includes such activities as:

- Update `setup.py` with non-example information
- Arrange into appropriate folders, such as `src` and `test`
- Update the `codeowners` to reflect subject matter expertise and codebase familiarity
  - Aim to have at least two codeowners for every repository

Some skeletons need additional configuration, such as with `skeleton-python-library` and its module structure inside `src/example`.

Run pre-commit against existing files

The skeleton will bring along with it our standard pre-commit hook configurations, including linting and other checks, with `setup-env`.

```
# Check all existing files
pre-commit run --all-files
```

The linters will automatically fix files where it can, however you are probably looking at a long list of updates to make before automated checks will pass. You may want to send this output to a file to make it easier to review, e.g. `pre-commit run --all-files > fixme.txt`.

isort

For our `skeleton-python-library` example, you'll need to do some configuration with `isort` and `.isort.cfg` to deconflict packages.

- Remove known-first-party and known-third-party packages so the tool will auto-populate them during the `pre-commit` step.
- Manually add your package name (i.e. in `src`) as known-first-party.

Coveralls

If the repository needs coverage checks and integration with Coveralls:

- Modify the `.coveragerc` to point to the src package
- Add appropriate secrets so they're available to the Actions workflow, e.g. add a token from Coveralls to the repository's secrets as `secrets.COVERALLS_REPO_TOKEN` for the repo badge.

pytest

For Python projects, run `pytest` manually to verify that your newly-updated repository still passes its test suite.

Make a pull request

Once you've run through the configuration and testing stages, you've probably accumulated a number of commits on your `skeletonize` branch.

The next step is to make a pull request and have the team perform code reviews.